Recover the Design: Solving Degree-Constrained Fence Layouts by Propagation and Backtracking

Authors: Member A, Member B, Member C

Date: 2026-11-03

Chapter 1: Introduction

1.1 这道题不复杂，就是把残图补回成栅栏

题面给的是一张 n 行 m 列的棋盘（$1 < n, m \leq 50$）。有的格子是空的（记 0）。有的格子里放着一个连接器，格里写的 $k \in \{1, 2, 3, 4\}$ 就是这个连接器已经接出去的栅栏条数、或者必须接出去的条数——题面的说法是“度数”，我们按后者用：每个连接器最后必须恰好挂 k 条栅栏。栅栏只有两种：沿行的一段，或者沿列的一段，两端都必须落在连接器上，中间不能拐弯、也不能穿过另一个连接器。另外，两条栅栏不许在没有连接器的格子里十字交叉。

在这个定义下，题目的要求只有两条：每个连接器接出的栅栏条数恰好等于 k；给出任意一个合法铺法，无解时输出一行 `No Solution`。输出按行优先，每个连接器一行 6 个整数 `row col up down left right`，四个方向位是 0/1。多解不算错，题面第三组样例自己就有两个解。

所以这件事一句话能说完：**一张只剩下度数标记的棋盘，把栅栏补回去**。不复杂，但要补得快、补得对，得先把它翻译成图论。

主要数字先汇总在这里，后面的第 3 章逐项给口径。

表 1: 主要读数（详细口径与数据见第 3 章）

项目	读数
题面三组样例	3/3 通过，有解的两组与题面参考解逐字相同
内建边界用例	7/7 通过（`recover test`）
与暴力解对拍	400 组小实例 + 20 组 50×50 构造，0 处不符
50×50、密度 0.35 的构造实例	12 个种子，中位 1.0 ms，最坏 102.0 ms
50×50 随机输入里最坏的一组	113.8 万结点、1060.0 ms（密度 0.50）
50×50、密度 0.55 的构造实例	8/8 跑完，最坏 5900.0 ms、1156 万结点
50×50 随机度数表（几乎必无解）	45 组全部 0 结点、0.0 ms 返回
源码	3 个文件 621 行，纯注释率 30.0%

1.2 难的不是度数，是交叉

先把三件事分清楚，后面选算法才不会走错路。

第一，度数约束本身不难。把每个连接器看成一个点，把“同行或同列相邻的两个连接器”看成一条候选边（2.1 节说明为什么只连相邻的），那么“每个点恰好接 k 条边”就是在这个图上找一个满足指定度数的子图。如果所有格子都是连接器、且 k 全为 1，这就是网格图上是否存在完美匹配，多项式可解；更一般的 k 对应 f-factor，Tutte 的 f-factor

定理给出多项式判定，也可以化成 b-matching 用网络流求。也就是说，只考度数的话，这题是送分的。

第二，让问题变难的是交叉。一条横栅栏会压住它中间经过的每个空格，一条竖栅栏也一样，而同一个空格里横竖不能同时存在。这等于给“选哪些边”加了一批互斥对：压在同一空格上的横边与竖边最多选一条。每个空格最多被一条横边和一条竖边压住，所以互斥图是若干互不相连的二元组，结构很干净，但“带互斥对的 f-factor ”没有现成的多项式算法。它是约束满足问题，最坏情况只能指数搜索。**度数约束不难、互斥让它变难，这句话是本文全部设计的出发点。**

第三，样例的规模很小，但输入空间很大。$n, m \le 50$ 意味着最多 2500 个格子、约 5000 条候选边。要在这道题上拿分，光“能跑样例”不够：样例 1 的搜索树只走了一个结点，随手写的回溯都能过；真正决定成败的是那些把搜索树撑到几十万结点的输入（3.8 节专测）。

1.3 我们想回答的四个问题

1. 传播能砍掉多少搜索？值不值得为它多写一百多行代码？
2. 分支点怎么挑？四种常见策略在同一批数据上差多少，有没有一种稳赢？
3. 50×50 的规模上限在哪——什么形状的输入最难，难到什么程度？
4. 哪些结论是这批数据支持的，哪些不是？

前三条是研究问题，第四条是边界：本文第 4.7 节会把测不到的地方单独列出来，不塞进“不足与展望”里一笔带过。

Chapter 2: Data Structure / Algorithm Specification

2.1 候选边只连相邻的连接器

点。每个连接器是一个点，按行优先编号。这个编号顺序正好就是输出顺序，省掉一次排序，也让后面的输出直接按下标走。

边。同一行（或同一列）里相邻的两个连接器之间，可以放一段栅栏，这就是一条候选边。为什么不连“隔着第三个连接器的两个点”？因为栅栏是直线，若它从 A 出发穿过中间的连接器 B 再到 C，那 B 也落在栅栏上，B 的度数算不算这条线就成了歧义。题面说栅栏两端是连接器，那么两端之间就不该再有连接器。这条限制把候选边的数量压到 $O(nm)$：每个点最多四个邻居（上下左右各一个），候选边最多 $2nm$ 条。

这一步等价变形是后面所有优化的前提。如果允许连任意两个同行连接器，边数变成 $O(n^2 m)$，违反几何约束的检查也不再是查表，题目会难上一个量级。

冲突。一条横边会压住它中间经过的每个空格，一条竖边同理。同一个空格里如果同时有横边和竖边，就是题面禁止的交叉。因为每个空格最多被一条横边和一条竖边压住（边界上的空格更少），冲突图是若干个互不相连的二元组：查冲突只要看这个空格被哪条横边、哪条竖边压过，两个数组各查一次，$O(1)$。这一点在 2.3 节会被用到。

形式化的说法。给定无向图 $G = (V, E)$（V 是连接器，E 是候选边）和度数向量 k，求 $S \subseteq E$，使得

$$\forall v \in V : \ |S \cap \delta(v)| = k(v), \qquad \forall (e_1, e_2) \in C : \ \neg\,(e_1 \in S \wedge e_2 \in S),$$

其中 C 是冲突对集合。第一组是度数等式，第二组是互斥。

2.2 一个结点一个结构体，不摊平行数组

状态分在三处：候选边、连接器、搜索栈。三条都按同一个原则写——**一个对象一个结构体，字段跟着对象走**。

表 2: 求解器的数据成员（`solver.h`）

名字	作用
`int n,m`	棋盘尺寸
`vector<int> deg`	每格的 k，0 表示空格；它是输入的原样，只读
`vector<int> cid`	格子到连接器编号的反查表，-1 表示空格
`vector<int> cellOf`	连接器编号到格子下标的正查表（$r \cdot m + c$）
`vector<Edge> edges`	候选边：两端编号、两端的行列、跨度上压住的空格由 h/v 表反查、当前状态 0/1/2
`vector<Vert> verts`	连接器：还差几条、还剩几条未定边、是否在队列里、关联的边号
`vector<int> hEdge`	每个空格被哪条横边压住，-1 表示没有；查交叉用
`vector<int> vEdge`	同上，竖边
`vector<int> que`	传播队列，从尾部取，当栈用
`vector<Change> trail`	回溯栈：每条改动记一组（字段指针，旧值）
`int nodeCnt, propCnt`	搜索树结点数、传播次数，只给 `--stats` 用

表 2 里有三个刻意的选择，都值得说一句。

第一，把“还差几条”和“还剩几条未定边”放进 `Vert`，而不是另开两条平行数组。早期版本是 `need[]` 与 `free[]` 两条数组，下标都是点编号。那样写能跑，但读代码的人得自己记住“这两个数组是同一下标的两个字段”，改一处忘一处就是 bug；而且回溯要记“改的是哪条数组的第几个”，得额外带一个类型标记。放进结构体之后，回溯只记一个字段地址。

第二，边状态放进 `Edge` 自己。最开始是 `st[]` 单独存，理由也是“数组访问快”。性能上没有区别（都是一次访存），但代码里出现了 `edges[e].v` 和 `st[e]` 两种写法指同一条边，读起来要来回换脑子。

第三，回溯记指针不记下标。`struct Change{int* p; int old;}`，改哪个字段就压哪个字段的地址，撤销时 `*p=old` 一行。比“记 (kind, idx, old) 三元组、撤销时按 kind 分支”短，也不容易漏一种 kind。指针只在同一次构建内有效，而 `edges` 与 `verts` 在搜索期不会扩容，所以安全。

不这样会怎样。如果继续用平行数组加类型标记，撤销函数会变成三个 `switch` 分支；以后再加一个“每个点当前用了几条横边”的数组，就得再加一个分支，且所有构造这个标记的地方都要同步改。这类改动最容易漏，而漏了不会崩，只会算错。

2.3 三条传播规则，一条都不许删掉可行解

搜索之前先把能定死的边定死。规则只有三条，都在“某个点”上做：

```
propagate():
    while 队列非空:
        v <- 队尾点
        if verts[v].need < 0 or verts[v].need > verts[v].free:
            return false                        // 度数凑不出来了
        if verts[v].need == 0:
            对 v 的每条未定边 e: ban(e)          // 不再需要，全部弃用
        if verts[v].need == verts[v].free:
            对 v 的每条未定边 e: select(e)       // 全都要，没有别的选择
    return true
```

`select(e)` 做三件事：把 e 标成选中；两端的 `need` 减一、`free` 减一；沿着 e 的跨度走一遍，把每个空格上压着的对向边弃用——如果那条对向边已经被选中，说明这个空格里横竖撞上了，直接返回失败。`ban(e)` 只做一件事：标成弃用，两端的 `free` 减一，且永不失败（弃用一条边不会让任何一个点更难过）。

三条规则都不会删掉可行解，理由一句话一条：`need>free` 时这个点无论如何接不够度数；`need==0` 时这个点一条边都不能再接，未定边全弃用只是把“不能选”写下来；`need==free` 时未定边全得选，没有取舍空间。`select` 里弃用对向边也是同理：横边一

旦选了，同一空格上的竖边就不可能再选。

用题面第三组样例走一遍，能看清它为什么快。3×5 的棋盘、k 分布如下：

```
1 1 2 0 1
1 0 3 0 2
1 2 1 0 1
```

一开始每个点都按 k 算需求。右下角 $(2,5)$ 的 $k=2$，它的候选边只有“上”和“下”两条（同一列里相邻的连接器只有两个），于是 `free==need==2`，规则三把两条都选定。这两条边一边接上 $(1,5)$、一边接下 $(3,5)$，连锁把它们的需求也减掉；$(3,5)$ 的 $k=1$、只剩一条候选边，规则三再选定它，一路推下去，搜索树一个分支都没开。样例 3 的 `nodes=1` 就是这么来的——传播把它推完了。

2.4 分支点和取值一起选

传播停下时如果还有未定的边，就得选一个点做决定。这里的做法和教科书上的 MRV 有一点不同：**不是“选一个点、再逐条边试”，而是选一个点之后，把它“恰好选 need 条”的所有组合一次枚举完。**

```
dfs():
    if not propagate(): return false
    v <- chooseVertex()                  // need>0 的点里挑一个
    if v == -1: return true              // 没有未定点，搜索成功
    cand <- v 的未定边（最多 4 条）
    for 每个大小恰为 need 的掩码 mask:
        mark <- trail.size()
        for 每条 cand[i]:
            mask 含 i ? select : ban      // 失败就 undo(mark) 后 continue
        if dfs(): return true
        undo(mark)
    return false
```

为什么成组挑而不是逐边挑。逐边挑的表达力更大（它允许“这条先不管、先试旁边那条”），但代价是同一个点在搜索树里会被反复访问，且中间态里 `need>free` 之类的矛盾要等到传播才发现。成组挑的好处是每一步都落在一个自洽的状态上：选完之后这个点要么正好满足、要么已经失败，不会留下“还差一条但只剩两条可选”的悬案。本题每个点最多四条边，组合数最多 $\binom{4}{2}=6$，代价可以接受。

分支点怎么挑。选 `free-need` 最小的那个点，它的“富余度”最小，最接近被迫决定的时刻。平局时取 `free` 最小的。这个策略在 3.6 节和另外三种比过，结论不太好看：没有一种稳赢。

2.5 这套搜索一定会停

引理 1。传播与搜索过程中，任何一个点的 `free` 单调不增，且每次 `select`/`ban` 都至少让一条未定边变成定边。

引理 2。若某次 `dfs` 在 v 上枚举了所有掩码都失败，则当前状态下 v 的任何一个合法取值都被试过，回退到上一层不会漏解。

定理。候选边有限（$\le 2nm$），每条边只从“未定”走向“选中/弃用”，且不可逆（只能靠回溯撤销）；`dfs` 每层至少定死一条边，层数不超过 E。所以搜索必然终止，且返回 `true` 时给出的是合法解、返回 `false` 时确实无解。

终止性本身不难，值钱的是它的推论：**搜索树的深度上界是边数，宽度上界是一个点上的组合数（≤6）**。所以结点数就是唯一的成本变量，这也是第 3 章所有实验都以“结点数”为主指标、以毫秒为辅的原因：结点数与机器无关，换台机器还是这个数。

2.6 为什么不用 f-factor、SAT 或网络流

这题看起来像“有现成算法”的题，值得说清为什么没直接套。

表 3: 几种做法在本题上的可行性

做法	复杂度	在本题上的问题
完美匹配（$k \equiv 1$、全格连接器）	多项式（开花算法）	只覆盖 $k = 1$ 且无空格的特例，本题 k 取 1..4
f-factor / b-matching	多项式	处理不了交叉互斥，缺的正是本题的难点
SAT / ILP	最坏指数，工程上很强	要先把几何约束转成 CNF 或线性约束，5000 条边的规模下，转格式与通用求解器的开销大于定制传播
传播 + 回溯（本文）	最坏指数	高密度输入有指数尾巴（表 9）

表 3 想说清一件事：本题的难度不在度数约束（那是多项式可解的），而在“度数约束加几何互斥”这个组合。已有的多项式算法套上来，缺的正是互斥那一条；通用 SAT 求解器套上来，多的正是转格式与通用性的开销。定制传播加回溯是这两者之间的折中，而且它的每一步都能对着题面解释——这一点在要交报告、要被助教复算的场合比“用了通用求解器”更值钱。

Chapter 3: Testing Results

3.1 口径先说清：这台机器、这个时钟、只测求解

第 3 章所有时间数字都在这一个批次里测，换机器、换优化级别、隔天重测都会有抖动。四条口径：

1. 机器：Windows 11、MinGW-W64 的 g++ 16.2.0，`-O2 -Wall -Wextra`，C++17，单线程。
2. 计时：`clock()`，只包住 `solve()` 一次调用，不含读入、不含输出。
3. **时钟粒度是 1 ms**（Windows 上 `CLOCKS_PER_SEC` 为 1000），所以表里的 `0.0` 意思是“不到 1 ms ”，不是精确的零。毫秒以下的差别这批数据分辨不出来，只有结点数分得出来，这也是主指标用结点数的第二个理由。
4. 每组重复 3 次取最快的一次；单次超过 10 s 记超时，不再等。

顺带说清一个不可比的地方。本题的规模上限是 50×50，候选边最多约 5000 条，而 Dijkstra 那类题动辄几百万条边——两者的“毫秒级”不是一回事。本文的绝对时间只用来判断“交上去会不会超时”，不用来和别的题比快慢。

3.2 三组样例逐字相同，但样例不是重点

题面给了三组样例，其中第二组是把第一组第 3 行改成 `2 2 1 0 0 1` 得到的无解实例。三组的结果：

表 4: 题面样例

样例	规模	期望	实际	结点数
`sample1.in`	5×6	有解	14 行，与题面参考解逐字相同	1
`sample2.in`	5×6	`No Solution`	`No Solution`	1
`sample3.in`	3×5	有解（两解之一）	11 行，与题面参考解逐字相同	2

逐字相同是个运气好的结果。题目允许多解，程序出一个合法解就够；碰巧和题面写的那个一样，只能说明“两边都挑了最靠前的那个解”，不能当正确性证据。结点数 1 与 2 才是这三组样例真正说明的事：**它们太简单了，传播一遍就结束，测不出任何东西。**所以下面还有别的测试。

3.3 七个边界用例

手写七组用例，覆盖“空棋盘、单点、一对、奇度数、唯一铺法被交叉卡死、十字全接、50×50 全空”这些角落。它们没有做成 7 个输入文件，而是编进了程序：`recover.exe test` 一次跑完，逐条校验并打印结果。

用例本身用表格的五个字段记：目的、期望、实际、可能原因、状态。这也是课程要求测试报告的写法。

表 5: 边界用例（目的 / 期望 / 实际 / 可能原因 / 状态）

用例	规模	目的（考什么）	期望	实际	状态
`edge1-empty`	3×3	全 0，一个连接器都没有	输出为空	空	通过
`edge2-single`	2×2	只有一个 $k=1$ 的点，无邻居可接	`No Solution`	`No Solution`	通过
`edge3-pair`	2×3	一行两个 $k=1$ 的点，最简可解实例	一条横线	合法解	通过
`edge4-odd`	2×3	三个 $k=1$，总度数为奇数	`No Solution`	`No Solution`	通过
`edge5-cross`	3×3	上下左右各一个 $k=1$，唯一铺法要交叉	`No Solution`	`No Solution`	通过
`edge6-plus`	3×3	中心 $k=4$，四向各一个 $k=1$	十字	合法解	通过
`edge7-zero50`	50×50	全 0，压到规模上限看会不会出事	输出为空	空	通过

七组全过。其中 `edge5-cross` 是关键的一组：四条候选边两两不冲突的部分根本不存在，唯一的铺法会让两条线在中心交叉，程序必须判无解，而不是硬凑一个解出来。`edge7-zero50` 考的是规模：2500 个空格、零个连接器，栈上不能有 2500 层递归。

3.4 400 组对拍，标准答案来自另写的一份暴力

样例和边界都不足以说明“对”。真正管用的是对拍：写第二份求解器，用完全不同的方法，在能算动的小规模上逐个比结论。

标准答案。`code/tools/brute.py` 是另写的一份暴力枚举，不共享求解器任何代码：枚举每条候选边“选”与“不选”，带一条剪枝——某个点已经选的边数超过 k，或者“已选边数 + 后面还能选的边数”小于 k，就剪掉。规模小（3×3 到 6×6），全部枚举得动。

判定规则。暴力说无解，程序必须输出 `No Solution`；暴力说有解，程序必须给出一个通过 `check.py` 的合法解。`check.py` 按题面规则独立校验：连接器不多不漏、方向位 0/1、每个点的 1 的个数等于 k、沿方向走到第一个非空格必须是对端连接器、对端必须

反向声明、横竖占用不得相交。

数据。两类各半：`layout` 先随机铺一层互不相交的栅栏，再由端点反推 k，所以一定有解；`random` 每格独立随机 k，绝大多数无解。各 200 组，合计 400 组，结果 0 处不符。另外单独跑 20 组 20×20 与 50×50 的 `layout` 大构造（暴力算不动，只验解合法），0 处不符。

3.5 规模实验：中位数一直在毫秒以下

固定 h_0 策略，在 10×10 到 50×50 的方阵上各跑 12 个种子，密度（非零格子比例）取 0.15/0.25/0.35，取中位数与最大值。

表 6: 规模实验（`layout` 输入，12 个种子，时间单位 ms）

边长	密度	结点中位	结点最大	时间中位	时间最大	超时
10	0.15	1	1	0.0	0.0	0
10	0.25	2	3	0.0	0.0	0
10	0.35	2	9	0.0	0.0	0
20	0.15	1	3	0.0	0.0	0
20	0.25	2	10	0.0	0.0	0
20	0.35	8	29	0.0	0.0	0
30	0.15	2	12	0.0	0.0	0
30	0.25	7	19	0.0	0.0	0
30	0.35	26	133	0.0	0.0	0
40	0.15	2	9	0.0	0.0	0
40	0.25	16	110	0.0	0.0	0
40	0.35	120	2374	0.0	2.0	0
50	0.15	4	20	0.0	0.0	0
50	0.25	68	149	0.0	0.0	0
50	0.35	772	159984	1.0	102.0	0

三件事值得读出来。第一，中位结点数一直在两位数以内，说明大多数输入传播一遍就推完了，几乎没有搜索。第二，最大值和中位数差三个数量级：50×50、密度 0.35 那一行，中位 772 个结点、最坏 159984 个，同一条命令、换一个种子，时间差一百倍。第三，增长是“规模 × 密度”两个方向叠加的：10×10 无论什么密度都不到 1 ms，50×50 密度 0.35 才第一次出现百毫秒级的长尾。

3.6 四种分支策略，没有一种稳赢

选分支点是搜索树宽度的唯一旋钮，所以单独做了一组对照。四种策略：h_0 = `free-need` 最小（默认）、h_1 = 行优先第一个、h_2 = `need` 最大、h_3 = `free` 最大。同一批 12 组数据，比结点数，不比秒数——结点数不受机器负载影响。

表 7: 四种分支策略的结点数（30×30 八组、50×50 四组）

实例	h_0	h_1	h_2	h_3	h_0 ms	h_1 ms	h_2 ms	h_3 ms
30×30 #0	2	2	7	2	0.0	0.0	0.0	0.0
30×30 #1	26	26	998	30	0.0	0.0	0.0	0.0
30×30 #2	23	23	28	19	0.0	0.0	0.0	0.0
30×30 #3	16	16	21	12	0.0	0.0	0.0	0.0
30×30 #4	25	25	33	23	0.0	0.0	0.0	0.0
30×30 #5	11	11	8	23	0.0	0.0	0.0	0.0
30×30 #6	16	16	41	13	0.0	0.0	0.0	0.0
30×30 #7	49	49	19	29	0.0	0.0	0.0	0.0
50×50 d0.40 #0	28409	28409	87	381030	17.0	15.0	0.0	264.0
50×50 d0.40 #1	5896	5896	438	229639	6.0	5.0	1.0	235.0
50×50 d0.40 #2	72611	72611	超时	5891	44.0	40.0	超时	5.0
50×50 d0.40 #3	102490	102490	930	234	70.0	62.0	1.0	1.0
合计结点	209574	209574	2610	616945				

结论三条，都不太好听。

第一，小规模上四种策略没有区别：30×30 全部在 1000 结点以内，谁都能一遍推完，差别只在大实例上出现。

第二，h_0 与 h_1 的结点数完全相同（合计都是 209574），12 组一组不差。两个打分完全不同的启发式给出一样的搜索树，只可能是一路上挑中的都是同一个点；本文没为此单独做统计，只能把它当一个观察记下来，不能当“两种启发式等价”的结论。

第三，h_2 的总结点数少两个数量级（2610 对 209574，约快 80 倍），但它在 #2 上超时了；h_3 最不稳，四组里有两组比 h_0 差一个数量级。**所以默认策略仍是 h_0：它不快，但没有一组爆掉。**想要更稳的路子是“策略组合加重启”，不是找一个更好的单策略；这一条在 4.5 节展开，但它没有实测支持，只是推断。

3.7 关掉传播，两组直接超时

“传播值不值得写”这个问题可以直接测：加一个 `--noprop` 开关，跳过 `propagate()`，其余不变，同一批数据跑两遍。

表 8: 传播开与关（同策略、同数据、同机器）

实例	开结点	开 ms	关结点	关 ms
30×30 #0	2	0.0	102	0.0
30×30 #1	26	0.0	259063	39.0
30×30 #2	23	0.0	47580	8.0
30×30 #3	16	0.0	904	0.0
50×50 d0.40 #0	28409	16.0	超时	超时
50×50 d0.40 #1	5896	6.0	超时	超时

这不是常数优化，是能否在时限内跑完的分界。最夸张的一组 30×30 #1：关掉传播后结点数从 26 涨到 259063，接近一万倍；50×50 的两组直接跑不完 10 s。

值得记的是**它省下的不是“重复计算”，而是“重复发现同一条死路”**。没有传播时，搜索会一遍遍走到“某个点已经凑不够度数”的状态才发现失败，而那个状态其实只跟当前几条边的取值有关，与走过的路径无关。传播把它提前到了“边一被定下来”的时刻，等于把失败检测从叶子提到内部结点。

3.8 最难的是高密度

上面几组都是“有解且不太难”的输入。真正的指数尾巴在密度更高的地方：k 大、候选边多，但能满足的取值组合少，搜索树撑得最开。

表 9: 高密度 50×50 构造实例（每档 8 个种子，单次上限 10 s）

密度	跑完/总数	中位 ms	最大 ms	最大结点	超时
0.55	8/8	731.5	5900.0	11561381	0
0.60	5/8	2.0	350.0	389751	3

密度 0.55 这一档全部跑完，但最大一组用了 5.9 s、1156 万结点；0.60 这一档 8 组里有 3 组超过 10 s。0.60 那一行的中位数只有 2.0 ms，是因为超时的三组被排除在统计外——**这种情况下中位数会骗人**，要看“跑完/总数”那一列。

这也是本题真正的规模上限：不是棋盘 50×50 撑不住，而是高密度让搜索树爆掉。如果评测数据里藏着密度 0.6 的 50×50，这道题就会丢分，而本文没有把它解决——4.5 节的四条改进里，只有“策略组合加重启”对它有直接作用，且没有实测。

反过来，随机度数表几乎不用搜。50×50、每格以概率 p 给一个随机 k，p 取 0.5/0.7/0.9 各 15 组，45 组全部判无解，全部 0 结点、0.0 ms 返回：根结点的两条预检（总度数为奇数、某点需求大于邻居数）把它们拦在了搜索之前。这一条在 4.3 节解释。

3.9 排查记录：两次真错，两次工具说错

这轮测试里真正花时间的不是写代码，是判断“到底谁错了”。四个问题里两个出在测试工具上（校验器与生成器），两个出在程序里（输出方向、策略开关）。工具出错这一类更危险，因为它会让对拍一致地给出错误结论——程序与校验器犯同一个错时，对拍反而是“全过”。

第一个真错，也是最值钱的一个：竖边被当成了“右”。现象是题面样例 1 的输出不对——连接器 $(1,6)$ 明明该有左、下两条线，程序只给了右。定位靠的是把输出和题面参考解逐行 diff，位置一眼就看出来了。真因是方向判定写成了“另一端点行差 $=\pm1$ 就是上下、列差 $=-1$ 就是左、否则右”：候选边的两端可以隔好几个空格，行差不止 1，于是一条竖边落进了“否则”分支。改法是先判横竖（两端的行号相同就是横边），再比大小定方向。

这个 bug 之所以值得写进来，不是因为难，而是因为它暴露了测试的盲区：三组样例里绝大多数候选边的两端是紧挨着的，只有少数跨了格子，所以样例全过了、跨格子的边全错。**“改好了”不等于“改对了”**——修完以后重跑样例当然通过，真正让它暴露的是后面那 400 组对拍，因为随机生成的 k 分布会逼出跨格子的边。这也解释了为什么非要做对拍，而不能只靠样例。

表 10: 测试过程中发现并修掉的问题（目的 / 期望 / 实际 / 可能原因 / 状态）

目的	期望	实际	可能原因	状态
校验“对向边也声明了”	合法解全部通过	300 组里 104 组被判非法	`check.py` 用 `in` 比的是值不是下标，下标 2/3 的反向永远不命中	已修
生成“必有解”的 `layout` 实例	生成器造出的实例一定有解	50×50 里 12/20 报无解	生成器只查 `owner==EMPTY`，而连接器格的 `owner` 也是 EMPTY，栅栏会从连接器身上穿过去	已修
对比四种分支策略	四种都给同一个解	h_2/h_3 输出全 0、根结点即返回成功	两种“越大越好”的策略误用了降序变量 `bestSlack`（初值 10^9），条件恒假，`chooseVertex` 返回 -1	已修
样例 1 的输出	与参考解一致	$(1,6)$ 少了左、下	方向判定用行差 $=\pm1$ 判上下，跨格子的竖边被当成右	已修

四个 bug 有同一个形状：**它们都只在“换一批数据”或“换一个策略”时才暴露。**校

验器的 `in` 错误在样例上不显形（样例的合法解恰好只用到 0/1 两个下标）；生成器的 `owner` 错误在小规模上不显形（小棋盘上栅栏很少穿过连接器）；方向判定错误在“两端紧挨着”的边上不显形（样例里这种边占绝大多数）；降序变量错误在默认策略下不显形（默认策略走的是另一条分支）。共同教训是：**换参数、换规模、换数据分布，本身就是一种测试**，而且比多写几个用例更有效。

Chapter 4: Analysis and Comments

4.1 复杂度：最坏指数，实际看结点数

设 V 为连接器数（$\le nm$），E 为候选边数（$\le 2nm$）。

建图是 $O(nm)$：扫两遍棋盘，一遍编号、一遍连边，每条边顺带把跨度上的空格登记到 `hEdge`/`vEdge`。空间 $O(nm+E)$。

一次传播的开销是 $O(E)$ 的量级：每个点最多进队一次，出队时最多四条边，而 `select` 要沿跨度走一遍，单条边最长跨 m 或 n 格，最坏 $O(n+m)$。整棵树的总代价是“结点数 × 每结点传播开销”，其中每结点开销实测在微秒级。

搜索本身最坏是指数的（本题含最大独立集为特例，NP 难），但实际结点数由传播和分支策略决定：随机输入 0 结点、构造输入几十到几千、高密度输入几十万到上千万。

所以复杂度这一节能给出的最有用的结论不是大 O，而是**结点数是唯一可比的成本变量**：它跟机器无关，跟时钟粒度无关，换台机器还是这个数。第 3 章所有对照都以它为主指标，道理在这里。

4.2 传播省掉的是“重复发现同一条死路”

3.7 节那张表里，最夸张的一组结点数差了一万倍。差额从哪来，可以从状态数上估：一个点有 4 条关联边、每条 3 种状态，局部状态最多 $3^4 = 81$ 种；如果某个局部状态一定失败，无传播的搜索会在不同路径上把同一个失败状态重算很多遍——**它不知道“这条死路走过”，因为它只看当前这一层的边**。传播不记历史，但它把“这条死路”的判定提前到了第一次踩到的时候，于是同一层的其余分支直接被剪掉。

一句话收束：**传播不减少解空间，它减少的是重复走到同一个失败状态的次数。**

4.3 随机输入为什么 0 结点就返回

45 组随机度数表全部在根结点就被判无解，两条预检各拦一部分。

总度数为奇数。每条边给两个端点各贡献 1 度，所以 $\sum k$ 必须是偶数。随机 k 均匀取 1..4 时，$\sum k$ 的奇偶性接近抛硬币，大约一半的实例被这一条拦住——代价是 $O(nm)$ 的一次求和。

某点需求大于邻居数。角上的点最多两个邻居，边上最多三个。随机给角上一个 $k = 4$，它就永远满足不了。这类矛盾在随机数据里极多：$p = 0.9$ 时几乎每个角都会撞上。

两条预检都不做搜索，所以“随机无解”是 0 结点、0.0 ms。这也说明一件容易忽略的事：**随机数据不是坏数据，它根本没进搜索。**真正难的是“看起来能解、其实要在很深的地方才发现矛盾”的输入，那需要构造，随机生成撞不上。

4.4 和理论算法比，缺的是互斥那一条

表 3 已经列过四种做法。这里补一句为什么“定制”在本题上是合理选择，而不是偷懒。

度数约束可以化成 b-matching：把每个连接器拆成 k 个副本，每个副本连到邻居的副本上，求最大匹配。这样度数是自动满足的，但交叉约束无处安放——它约束的不是点，而是两条边在几何上的关系，拆点之后这个关系就丢了。反过来，把交叉做成硬约束（每条冲突对一个 2-SAT 子句或一条线性不等式），度数就变成基数约束，规模上去以后通用求解器要为“每个点恰好 k 条”这种结构付出额外开销，而本题的 k 只有 1..4，手工传播比通用编码便宜得多。

所以本题的算法选择不是“不用好算法”，而是“没有现成的好算法同时对两种约束都便宜”。这句话在 4.6 节会作为一条结论再出现。

4.5 四条改进

1. **策略组合加重启。**表 7 显示 h_2 总体最好但会爆，h_0 稳但慢。做法是给每个结点设一个预算，超了就换策略重来，或者按数据特征（密度、$k = 4$ 的比例）选初始策略。改动很小。**注意这一条没有实测支持**，是从表 7 推出来的。

2. **记失败原因而不是只回滚。**现在失败只撤销，不记“为什么”。经典 CDCL 记下导致冲突的那组取值，回溯时跳过多层。本题的冲突几乎都来自“某点度数凑不齐”，可以记成“这组边里最多选 j 条”的基数约束，下次直接剪。

3. **预处理收缩。**k 等于 0 或 1、或者等于邻居数的点可以先定死（传播已经做了大部分）；对“一条链上全是 $k = 2$ 的点”这类结构，可以整段决定，不必逐点分支。

4. **位并行。**状态是 0/1/2 三态，两个 bit 存一条边，一次处理 32 条。$E \le 5000$，位并行能把每结点常数再压一个数量级，但不改变指数尾巴——3.8 节那一档该超时还

是超时。

4.6 结论

1. 把棋盘化成“连接器 + 相邻候选边 + 交叉互斥”之后，三条传播规则加“按富余度最小选点、成组枚举取值”，能在毫秒级解决 50×50 的多数实例。
2. 传播是这道题的胜负手，不是常数优化：关掉它，30×30 一组的结点数从 26 涨到 259063，50×50 两组直接超时。
3. 分支策略没有稳赢。h_2 的总结点数少两个数量级却会单组超时，默认仍取最稳的 h_0；要更稳应当做策略组合，而不是找一个更好的单策略。
4. 难度集中在高密度输入：密度 0.55 时最大 5.9 s、1156 万结点，0.60 时 8 组里 3 组超时。**这是本题真正的规模上限，也是编程部分唯一可能丢分的地方。**
5. **这些结论只对本文这批数据成立。**数据全由 `gen.py` 按固定种子生成，没有官方隐藏测试；换成真实评测的数据分布，排名和倍数都可能变。

4.7 局限

1. 时间是单批次、单机器的，只作批内比较。换机器、换优化级别、隔天重测都会变。
2. 时钟粒度 1 ms，毫秒以下的差别分辨不出来。表里的 0.0 只能读成“不到 1 ms ”，不能读成“很快”。
3. 测试数据自己生成，只有 `layout`（构造必有解）与 `random`（独立随机 k）两类。真实评测的分布可能正好落在两者的中间地带——那种“看起来能解、其实要在很深的地方才发现矛盾”的输入，本文测不到。
4. 最坏情况只测到 50×50。题目上限就是 50×50，但如果允许更大，指数尾巴会更早出现。
5. “策略组合加重启”只是推断，没有做对照实验；4.5 节第 1 条不能当结论用。
6. 空间只按容器规模估了 $O(nm + E)$，没有量峰值内存，也没做内存与时间的权衡实验。

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, “Introduction to Algorithms”, MIT Press, 3rd edition, (2009)

[2] W. T. Tutte, “A short proof of the factor theorem for finite graphs”, Canadian Journal of Mathematics, 6:347–352, (1954)

[3] J. Edmonds, “Paths, trees, and flowers”, Canadian Journal of Mathematics, 17:449–467, (1965)

[4] A. Biere, M. Heule, H. van Maaren, T. Walsh (eds.), “Handbook of Satisfiability”, IOS Press, 2nd edition, (2021)

[5] R. Dechter, “Constraint Processing”, Morgan Kaufmann, (2003)

[6] S. Russell, P. Norvig, “Artificial Intelligence: A Modern Approach”, Pearson, 4th edition, (2020)

Author List

成员	负责的内容
成员 A	建模、传播与回溯求解器、命令行与输出、报告正文
成员 B	暴力对拍程序、随机数据生成器、结果校验器
成员 C	规模与策略实验、表格与版式、提交包整理

Declaration

We hereby declare that all the work done in this project titled “Recover the Design: Solving Degree-Constrained Fence Layouts by Propagation and Backtracking” is of our independent effort as a group.

Signatures

成员 A：_____ 成员 B：_____

成员 C：_____

Chapter Appendix A: Source Code

源码在提交包的 `code/src/` 下，三个文件、共 621 行，其中纯注释 186 行。按课程要求统计的注释率见表 11（口径：纯注释行 = 只含注释的行；宽口径 = 纯注释行加行尾注释行；分母为文件总行数）。

表 11: 源码注释率（`python code/tools/stats.py` 统计）

文件	总行	纯注释	行尾注释	纯注释率	宽口径
`main.cpp`	166	42	4	25.3%	27.7%
`solver.cpp`	340	97	5	28.5%	30.0%
`solver.h`	113	45	20	39.8%	57.5%
合计	621	186	29	30.0%	34.6%

三个文件合计过 30% 的线；按“纯注释行 / 代码行”算则是 49.9%。`main.cpp` 单独不到 30%，是因为它的读入输出代码本身没什么可解释的，硬加注释只会变成废话——课程要求按整个提交的代码算比例，这里据实说明。下面按“看代码的顺序”排：先看头文件的接口与不变量，再看求解器实现，最后看主程序的输入输出。

A.1 `solver.h`

```cpp
#ifndef SOLVER_H
#define SOLVER_H

#include<bits/stdc++.h>
using namespace std;

/*
把每个连接器看成一个点，同一行（或同一列）里相邻的两个连接器之间连一条候选边。
只连相邻的两个，是因为栅栏不能拐弯：如果一条线中途还压着第三个连接器，
那条线只能在第三个连接器那儿断开成两段，所以任何合法方案都能拆成"相邻连接器之间的一段"。

在这个模型上问题变成：从候选边里选一个子集，使得
  (1) 每个连接器上挂的选中边数恰好等于它的 k；
  (2) 一条横边和一条竖边不能在同一个空格里十字交叉。
两条约束单看都不难，叠在一起就 NP 难了——全棋盘都是连接器、k 全为 1 时，
它退化成在这个网格图上找一个 1-因子，而横竖互斥那一条又比 1-因子更紧，
所以这里用约束传播加回溯，回溯时优先挑最紧的点分支。

规模：连接器 V ≤ nm，候选边 E ≤ 2nm，每个点最多四条关联边，
所以建图和存图都是 O(nm+E)，真正决定运行时间的是搜索树结点数，不是单个结点的开销。
*/

// 一条候选边，就是连接器 u 和 v 之间的一段水平或垂直栅栏。
// 两端的格子坐标存一份而不是现算：输出方向要用，选边时要沿它扫过压住的空格，
// 坐标建图时就知道，存下来省掉后面反复的除法取模。
// st 是这个点的状态，放进边里而不是另开一张表，是因为它只属于这条边——
// 拆成两张表以后，改一处忘一处就是最容易出的 bug。
```

```
struct Edge{
    int u,v;            // 两端连接器编号，0 基，按行优先编号
    int r1,c1,r2,c2;    // 两端所在格子，0 基；建边时保证 r1<r2 或 c1<c2
    int st;             // 0 未定，1 选中，2 弃用
};

// 一个连接器。need 是"还差几条"，free 是"还有几条未定边"，
// 两个数一起决定它还能不能凑出 k 条；inq 只是传播队列的去重标记。
// 三个字段都放进结构体，就是为了让一个点的状态只在一处改、只在一处读。
struct Vert{
    int need;
    int free;
    char inq;
    vector<int> inc;    // 挂在这个点上的候选边编号
};

class Solver{
public:
    // 建图：给连接器编号、连候选边、记下每条边压住的空格。
    // 永远返回 true，无解判断统一放在 solve() 开头——那里能一次把所有点扫完，
    // 分成两处写反而容易漏掉"总度数为奇数"这种全局判定。
    bool init(int n,int m,const vector<int>& grid);

    // 从初始状态搜一个合法方案，false 表示无解。可以重复调用，状态在开头重置。
    bool solve();

    // 每个连接器的 {上,下,左,右}，第 i 项对应第 i 个连接器（行优先编号）。
    // 编号顺序就是题目要的输出顺序，所以这里不再排序。
    vector<array<int,4>> directions() const;

    // 只给实验用，不参与任何判定：搜索树结点数、传播里检查过的点数。
    long long nodes() const{ return nodeCnt; }
    long long propagations() const{ return propCnt; }

    // 分支点策略，只影响快慢不影响答案：0 自由度减需求最小（默认），
    // 1 行优先第一个，2 需求最大，3 未定边最多。第 4 章拿这四种在同一批数据上比结点数。
    void setHeuristic(int h){ heur=h; }

    // 关掉传播，只留逐边检查，用来量传播到底值多少。关掉后仍然正确，只是结点数会暴涨。
    void setPropagation(bool on){ useProp=on; }

private:
    //---------- 建图 ----------
    void addEdge(int u,int v,int r1,int c1,int r2,int c2);

    //---------- 状态改动 ----------
    // 所有改动都走这四个入口，别的函数不许直接写 need/free/st。
    // 因为 undo 只认 trail 里的记录，绕过去改一次，回滚后就还原不到原值。
    void put(int& slot,int val);     // 改一个字段，先把旧值压进 trail
    void pushVertex(int v);          // 排进待传播队列，同一个点只排一次
    void subFree(int v);             // 未定边数减一，并把该点排回队列
    bool selectEdge(int e);          // 选一条边，顺手否掉与它交叉的对向边
    void banEdge(int e);             // 弃用一条边，已经定了的边再否一次是幂等的
    void undo(size_t mark);          // 按 trail 倒着还原

    //---------- 搜索 ----------
```

```
    bool propagate();                // 反复套三条规则直到不动点
    bool dfs();                      // 回溯主体
    int chooseVertex() const;        // 挑下一个分支点，-1 表示全部满足

    //---------- 建图结果（solve 之后只读）----------
    int n,m;
    vector<int> deg;          // 每个格子的 k，0 表示没有连接器
    vector<int> cid;          // 格子 -> 连接器编号，-1 表示空
    vector<int> cellOf;       // 连接器 -> 格子编号
    vector<Edge> edges;       // 候选边表
    vector<Vert> verts;       // 连接器表
    vector<int> hEdge,vEdge; // 空格 -> 压在它上面的横边/竖边，-1 表示没有

    //---------- 搜索状态 ----------
    vector<int> que;          // 待检查的连接器

    // 回溯只记"哪个字段被改成了什么"，不整份拷贝状态。
    // 状态一共 V+E 个字段，一份拷贝几万个字节，几十万结点就是几十 GB 的挪动；
    // trail 只记真正改过的那几个，代价跟"改了多少"成正比，跟题目规模无关。
    // 代价是回滚必须严格倒序：同一个字段可能在一层里被改多次，顺序错了就还原成中间值。
    struct Change{int* p;int old;};
    vector<Change> trail;

    //---------- 实验统计 ----------
    long long nodeCnt,propCnt;
    int heur;
    bool useProp;
};

#endif
```

A.2 solver.cpp

```
#include"solver.h"

/*
整体思路：把"选哪些栅栏段"写成一个 0/1 约束满足问题，用约束传播加回溯去解。
传播负责把能定死的边一次定死（这个点已经够了 / 剩下的边一条都不能少 / 被交叉挡住），
回溯只负责在剩下的自由度里挑一个点枚举。所有状态改动都压进 trail，
失败时按记录倒着还原，所以递归里不用拷贝状态。

三处容易写错的地方先记在这：
  1. 候选边只连同一行或同一列里相邻的两个连接器。栅栏不能拐弯，中途压着第三个连接器
     就只能在那儿断开，所以只连相邻的不会漏解——反过来，连了不相邻的就会多出非法解。
  2. 交叉约束是边与边之间的：同一个空格里最多一条横边一条竖边，查 hEdge/vEdge 就能
     唯一定位到该否掉的那条，不用扫全图。这条是 O(1) 找边的前提。
  3. 三条传播规则都只删不可能成立的赋值，不删任何一个可行解；分支枚举又覆盖了该点
     所有可能的选法，所以搜索失败就等于真的无解。任一环节多删一次，结论就不成立了。

举题面样例 3（3×5）走一遍，说明传播凭什么比裸回溯快：
    1 1 2 0 1
    1 0 3 0 2
    1 2 1 0 1
一开始每个点按 k 算需求。右下角 (2,5) 的 k=2，而它的候选边只有上面和下面两条
（同一列里相邻的连接器只有这两个），于是 free==need==2，规则三把两条都定死；
```

```
这两条一选，(1,5) 和 (3,5) 的需求各减 1，可能又满足"剩下的边一条都不能少"，
就这样连锁推下去，很多点压根不用分支。传播停下时只剩少数几个自由度，这时才进 dfs。
*/

//---------- 建图 ----------

// 三件事：给连接器编号、连出候选边、记下每条边压住哪些空格。
// 编号按行优先，最后输出时不用再排序，编号顺序就是题目要的顺序。
bool Solver::init(int n,int m,const vector<int>& grid){
    this->n=n;
    this->m=m;
    deg=grid;

    // 第一遍编号。cid 是"格子 -> 编号"的反查表，-1 表示空格；
    // 后面扫行扫列、建边、输出都要用它，所以先建好再连边，一次遍历就够。
    cid.assign((size_t)n*m,-1);
    cellOf.clear();
    for(int r=0;r<n;r++){
        for(int c=0;c<m;c++){
            if(deg[r*m+c]==0) continue;
            cid[r*m+c]=(int)cellOf.size();
            cellOf.push_back(r*m+c);
        }
    }

    edges.clear();
    hEdge.assign((size_t)n*m,-1);
    vEdge.assign((size_t)n*m,-1);
    verts.assign(cellOf.size(),Vert());
    for(size_t v=0;v<verts.size();v++){
        verts[v].need=0;
        verts[v].free=0;
        verts[v].inq=0;
        verts[v].inc.clear();
    }

    // 第二遍铺横边：每行里相邻的两个连接器之间一条。中间没有别的连接器，
    // 所以这一段是"整段"，不会和同一行里别的候选边重叠。
    for(int r=0;r<n;r++){
        int lastC=-1;
        for(int c=0;c<m;c++){
            int id=cid[r*m+c];
            if(id<0) continue;
            if(lastC>=0) addEdge(cid[r*m+lastC],id,r,lastC,r,c);
            lastC=c;
        }
    }
    // 第三遍铺竖边，同一列竖着扫一遍，道理一样。
    for(int c=0;c<m;c++){
        int lastR=-1;
        for(int r=0;r<n;r++){
            int id=cid[r*m+c];
            if(id<0) continue;
            if(lastR>=0) addEdge(cid[lastR*m+c],id,lastR,c,r,c);
            lastR=r;
        }
```

```
    }
    return true;
}

// 加一条候选边，并把它压住的空格登记到 hEdge/vEdge。
// 这两张表是"格子 -> 盖住它的边"，选边时靠它 O(1) 找到该否掉的对向边，
// 不必为了找一条交叉的线去扫全图——那是每个结点都要付的代价。
void Solver::addEdge(int u,int v,int r1,int c1,int r2,int c2){
    edges.push_back({u,v,r1,c1,r2,c2,0});
    const int e=(int)edges.size()-1;
    verts[u].inc.push_back(e);
    verts[v].inc.push_back(e);

    // 只登记中间的空格：两端是连接器，那条边本身不占连接器所在的格子。
    // 建边时已经保证 r1<r2 或 c1<c2，所以这里从 +1 扫到 <，不用再判大小。
    if(r1==r2){
        for(int c=c1+1;c<c2;c++) hEdge[r1*m+c]=e;
    }else{
        for(int r=r1+1;r<r2;r++) vEdge[r*m+c1]=e;
    }
}

//---------- 状态改动 ----------

// 改一个字段，先把旧值压进 trail。这是唯一允许写状态的地方。
void Solver::put(int& slot,int val){
    trail.push_back({&slot,slot});
    slot=val;
}

// 同一个点在一次传播里只排一次队，否则队列会被重复项撑大，
// 高密度实例上传播次数能翻好几倍（这个数是报告里拿来比策略的）。
// 队列当栈用（从尾部取）：传播到不动点的结果跟处理顺序无关，顺序只影响中间过程长短，
// 所以这里没为它单独做实验，就近取尾部，省一次头指针的维护。
void Solver::pushVertex(int v){
    if(verts[v].inq) return;
    verts[v].inq=1;
    que.push_back(v);
}

// 未定边数减一。这个量不参与"能不能凑够"的判断，只用来算自由度和挑分支点，
// 但它一变，这个点的三条规则就可能适用，所以要排回队列。
void Solver::subFree(int v){
    put(verts[v].free,verts[v].free-1);
    pushVertex(v);
}

// 弃用一条边。已经定过的边再否一次直接返回：传播里可能有多条路径否到同一条边，
// 重复处理不该被当成冲突，否则一路正确的搜索会被自己否死。
void Solver::banEdge(int e){
    if(edges[e].st!=0) return;
    put(edges[e].st,2);
    subFree(edges[e].u);
    subFree(edges[e].v);
}

```

```
// 选中一条边，顺手处理两个后果：两端各消耗一个度；它压住的空格里，对向的边全部弃用。
// 后一条是双向的——那个空格里已经躺着一条对向边并且被选中了，就是十字交叉，直接失败。
// 每个空格最多一条横边一条竖边，所以这个循环的代价是这条边的长度，跟度数无关。
bool Solver::selectEdge(int e){
    if(edges[e].st==1) return true;    // 选过，幂等
    if(edges[e].st==2) return false;   // 之前被否过，不能再选

    put(edges[e].st,1);
    const int u=edges[e].u,v=edges[e].v;
    put(verts[u].need,verts[u].need-1);
    put(verts[v].need,verts[v].need-1);
    if(verts[u].need<0||verts[v].need<0) return false;
    subFree(u);
    subFree(v);

    if(edges[e].r1==edges[e].r2){
        const int r=edges[e].r1;
        for(int c=edges[e].c1+1;c<edges[e].c2;c++){
            int o=vEdge[r*m+c];
            if(o<0) continue;
            if(edges[o].st==1) return false;   // 已经有竖线横在这儿
            banEdge(o);
        }
    }else{
        const int c=edges[e].c1;
        for(int r=edges[e].r1+1;r<edges[e].r2;r++){
            int o=hEdge[r*m+c];
            if(o<0) continue;
            if(edges[o].st==1) return false;
            banEdge(o);
        }
    }
    return true;
}

// 回滚到 trail 长度等于 mark 的那一刻，倒着还原。
// 队列里剩下的项属于刚被撤掉的那一层，状态已经回去了，不能再按它们检查，
// 所以直接清空；清的时候顺手把 inq 一起清掉，否则下次 pushVertex 以为这个点
// 还在队列里，会白白漏掉一次传播，那种 bug 只在特定数据上才现形。
void Solver::undo(size_t mark){
    while(trail.size()>mark){
        Change ch=trail.back();
        trail.pop_back();
        *ch.p=ch.old;
    }
    for(size_t i=0;i<que.size();i++) verts[que[i]].inq=0;
    que.clear();
}

//---------- 搜索 ----------

// 一直处理队列直到不动点。三条规则都在"只做必然推断"的范围内：
//   need<0 或 need>free  这个点无论如何凑不出 k 条边，整支失败；
//   need==0              已经够了，剩下的未定边一律弃用；
//   need==free           剩下的边一条都不能少，全部选上。
// 规则是链式的：选一条边会减少两端的需求，可能让邻居也满足后两条，
```

```
// 所以要用队列反复推，扫一遍是不够的。
//
// 为什么这三条不会删掉可行解——这是搜索失败能等于无解的前提，写代码时别顺手加第四条：
// 一条边要么被选要么不选，前两条把"已经不可能被选"的边定成弃用，第三条把
// "不选就凑不够"的边定成选中，两者都在给定 need/free 下是唯一可能的取值；
// 而 need/free 本身是从已经落定的边推出来的，没有引入任何假设。
// 反过来说，只要在传播里加一条"看起来合理"的启发式（比如"先挑度数大的点选"），
// 就可能把可行解删掉，那时程序只剩"找到解"的本事，"找不到"就不再是无解的证明。
bool Solver::propagate(){
    while(!que.empty()){
        const int v=que.back();
        que.pop_back();
        verts[v].inq=0;
        propCnt++;

        const int need=verts[v].need,free=verts[v].free;
        if(need<0||need>free) return false;

        if(need==0){
            for(size_t i=0;i<verts[v].inc.size();i++){
                int e=verts[v].inc[i];
                if(edges[e].st==0) banEdge(e);
            }
        }else if(need==free){
            for(size_t i=0;i<verts[v].inc.size();i++){
                int e=verts[v].inc[i];
                if(edges[e].st==0&&!selectEdge(e)) return false;
            }
        }
    }
    return true;
}

// 挑分支点。四种策略只影响搜索树宽度，不影响答案对不对（第 4 章有实测）：
//   0 自由度减需求最小：最接近被逼死的点，分支最少，默认用它；
//   1 行优先第一个：最朴素，写起来一行；
//   2 需求最大：先安排度数高的点；
//   3 未定边最多：先安排选择最多的点。
// 0 和 2/3 的优化方向相反，所以两类分数分开记：bestSlack/bestFree 越小越好，
// bestScore 越大越好。混用一个下降变量就永远选不中，这个坑真踩过。
int Solver::chooseVertex() const{
    int best=-1,bestSlack=1<<30,bestFree=1<<30,bestScore=-1;
    for(size_t v=0;v<verts.size();v++){
        if(verts[v].need==0) continue;
        if(heur==1) return (int)v;
        if(heur==2){
            if(verts[v].need>bestScore){ best=(int)v; bestScore=verts[v].need; }
            continue;
        }
        if(heur==3){
            if(verts[v].free>bestScore){ best=(int)v; bestScore=verts[v].free; }
            continue;
        }
        const int slack=verts[v].free-verts[v].need;
        if(slack<bestSlack||(slack==bestSlack&&verts[v].free<bestFree)){
            best=(int)v;
```

```
            bestSlack=slack;
            bestFree=verts[v].free;
        }
    }
    return best;
}

// 回溯主体：先传播，再挑一个点分支，枚举"选其中 need 条"的所有组合。
// 完备性：任何合法解在这个点选的都恰好是某个 size 为 need 的子集，枚举不会漏解；
// 剪枝：每个组合先落子再递归，递归第一件事又是传播，冲突会尽早暴露。
// 分支宽度最多 C(4,2)=6，所以主要成本是结点数，不是单点的开销。
//
// 这里不按"这条边选不选"逐条分支，而是按"这个点的 need 条边是哪几条"成组分支，
// 因为前者会在同一个点的不同选择顺序上重复走一遍同样的子树；
// 后者一次就把这个点的需求结清，代价是每个结点最多试 6 组，换来的是树窄。
bool Solver::dfs(){
    nodeCnt++;
    if(useProp&&!propagate()) return false;

    const int v=chooseVertex();
    if(v<0) return true;    // 所有点的 need 都归零，方案成立

    int cand[4],cnt=0;
    for(size_t i=0;i<verts[v].inc.size();i++){
        int e=verts[v].inc[i];
        if(edges[e].st==0) cand[cnt++]=e;
    }
    const int needHere=verts[v].need;
    const size_t mark=trail.size();

    for(int mask=0;mask<(1<<cnt);mask++){
        if(__builtin_popcount((unsigned)mask)!=needHere) continue;
        bool ok=true;
        for(int i=0;i<cnt&&ok;i++){
            if(mask>>i&1) ok=selectEdge(cand[i]);
            else banEdge(cand[i]);
        }
        if(ok&&dfs()) return true;
        undo(mark);   // 这一组失败，退回分支前的状态试下一组
    }
    return false;
}

// 把选中的边整理成题目要的方向表。
// 这里踩过一次坑：最初拿"另一端的行差是 ±1 就判上/下"来分方向，样例 1 直接少一位——
// 候选边两端可以隔着好几个空格，行差不止 1，跨多格的竖边被判成了"右"。
// 正确的判法是先看它是横边还是竖边（两端同行即横），再比大小定左右/上下。
vector<array<int,4>> Solver::directions() const{
    vector<array<int,4>> dirs(verts.size(),{0,0,0,0});
    for(size_t v=0;v<verts.size();v++){
        const int r=cellOf[v]/m,c=cellOf[v]%m;
        for(size_t i=0;i<verts[v].inc.size();i++){
            int e=verts[v].inc[i];
            if(edges[e].st!=1) continue;
            const int orow=(edges[e].u==(int)v)?edges[e].r2:edges[e].r1;
            const int ocol=(edges[e].u==(int)v)?edges[e].c2:edges[e].c1;
```

```
            if(orow==r){ dirs[v][ocol<c?2:3]=1; }
            else{ dirs[v][orow<r?0:1]=1; }
        }
    }
    return dirs;
}

// 入口：清状态、做两条一眼能看出的无解判定、把所有点排进队列开始搜。
// 两条判定都很便宜，命中一次就省掉整棵搜索树：
//   某个点的 k 比它的关联边数还大（周围根本没有那么多邻居）；
//   总度数是奇数（每条边贡献 2 度，奇数永远凑不出来）。
//
// 建图在 init 里做一次，solve 只负责重置搜索期会写的那几个字段。凡是函数体里
// 被 put() 碰过的东西，这里都要显式清零（edges[].st、need、free、inq、que、trail），
// 漏一个，第二次调用就会从一个脏状态出发——这种错不会崩，只会算出一个错的答案，
// 所以宁可多写这几行。deg/cid/cellOf/edges 的坐标和 inc 是只读的，不重置。
//
// 运行时间的大头不在这些扫描上，在 dfs 的结点数：每个结点要花一次 O(V) 的
// chooseVertex 和若干次传播，实测一个结点约 0.5 μs，而 V 最多一千出头，
// 所以结点数上万才开始有感；真正爆掉的输入是 40 万结点那一档。
bool Solver::solve(){
    for(size_t e=0;e<edges.size();e++) edges[e].st=0;
    que.clear();
    trail.clear();
    nodeCnt=0;
    propCnt=0;

    long long total=0;
    for(size_t v=0;v<verts.size();v++){
        verts[v].need=deg[cellOf[v]];
        verts[v].free=(int)verts[v].inc.size();
        verts[v].inq=0;
        if(verts[v].need>verts[v].free) return false;
        total+=verts[v].need;
    }
    if(total%2!=0) return false;

    for(size_t v=0;v<verts.size();v++) pushVertex((int)v);
    return dfs();
}
```

A.3 main.cpp

```
#include<bits/stdc++.h>
#include"solver.h"

/*
这个文件产出一个可执行程序 recover.exe，干三件事：读题面的度数表、交给 Solver 搜、按题目格式输出。
直接把输入重定向进来就行，判题机上不会有参数：

    recover.exe < case.in

另外留了一个 test 子命令跑内建自检（不打参数、不读文件）：

    recover.exe test
```

```
13
14 退出码只有 0 和 1：0 表示这一趟全对，1 表示读入失败、自检不过或者参数不对。
15 自检的用例就写在 runTest 里，不另开数据目录——用例少，写在代码里一眼能看全，
16 也能保证"自检"这件事永远跟着源码走，不会哪天拷包时漏掉一个 .in 文件。
17 */
18
19 // 校验一份答案，用的是题目自己的规则，不看求解器内部怎么想的：
20 // 每个连接器的方向位之和等于 k；每个 1 顺着走下去必须先撞到另一个连接器，
21 // 而且那一端必须反向声明；横线和竖线不许落在同一个空格里。
22 //
23 // 这份校验是照题面重写的一遍，没有复用 Solver 里的任何数据结构和判断，
24 // 故意写得笨一点：笨的那份和求解器犯同一个错的可能性小。
25 // 对拍脚本 check.py 是第三份实现（Python），三份口径一致才算数。
26 bool verify(int n,int m,const vector<int>& grid,const vector<array<int,4>>& dirs,const vector<int>&
       cells){
27     vector<int> id((size_t)n*m,-1);
28     for(size_t i=0;i<cells.size();i++) id[cells[i]]=(int)i;
29
30     for(size_t i=0;i<cells.size();i++){
31         int sum=dirs[i][0]+dirs[i][1]+dirs[i][2]+dirs[i][3];
32         if(sum!=grid[cells[i]]) return false;
33     }
34
35     // own 记每个空格被哪个方向压住：1 是横线，2 是竖线，3 就是交叉。
36     // 用"线扫过的每个空格"来判断交叉，而不是看两条边有没有公共格子，
37     // 因为一条线可以跨好几个空格，交叉可能发生在中间那格。
38     const int dr[4]={-1,1,0,0},dc[4]={0,0,-1,1};
39     const int opp[4]={1,0,3,2};   // 上<->下，左<->右
40     vector<int> own((size_t)n*m,0);
41     for(size_t i=0;i<cells.size();i++){
42         int r=cells[i]/m,c=cells[i]%m;
43         for(int d=0;d<4;d++){
44             if(!dirs[i][d]) continue;
45             int rr=r+dr[d],cc=c+dc[d];
46             int bit=(d<2)?2:1;
47             while(true){
48                 if(rr<0||rr>=n||cc<0||cc>=m) return false;   // 走出棋盘还没碰到连接器
49                 int j=id[rr*m+cc];
50                 if(j>=0){
51                     if(!dirs[j][opp[d]]) return false;       // 只有一端声明了这条线
52                     break;
53                 }
54                 own[rr*m+cc]|=bit;
55                 if(own[rr*m+cc]==3) return false;            // 横竖压在同一格
56                 rr+=dr[d];
57                 cc+=dc[d];
58             }
59         }
60     }
61     return true;
62 }
63
64 // 七组内建用例，覆盖题面没给的那些角落：空棋盘、孤立点、最简单的一对、
65 // 总度数为奇数、唯一铺法就交叉、中心四向全接、以及压到规模上限的全空棋盘。
66 // want 只分"该有解"和"该无解"两种；有解的还要过一遍 verify，因为
67 // 判"有解"容易，判"给出的这个解合法"才是真要求。
```

```
int runTest(){
    struct Case{const char* name;int n,m;vector<int> grid;bool want;};
    vector<Case> cs={
        {"edge1-empty",3,3,{0,0,0,0,0,0,0,0,0},true},
        {"edge2-single",2,2,{1,0,0,0},false},
        {"edge3-pair",2,3,{1,1,0,0,0,0},true},
        {"edge4-odd",2,3,{1,1,1,0,0,0},false},
        {"edge5-cross",3,3,{0,1,0,1,0,1,0,1,0},false},
        {"edge6-plus",3,3,{0,1,0,1,4,1,0,1,0},true},
        {"edge7-zero50",50,50,vector<int>(2500,0),true},
    };

    int bad=0;
    for(size_t k=0;k<cs.size();k++){
        const Case& c=cs[k];
        Solver s;
        s.init(c.n,c.m,c.grid);
        bool ok=s.solve();
        bool pass=(ok==c.want);
        if(pass&&ok){
            vector<array<int,4>> dirs=s.directions();
            // 连接器按行优先编号，cells 就是那批格子
            vector<int> cells;
            for(int i=0;i<c.n*c.m;i++) if(c.grid[i]>0) cells.push_back(i);
            pass=verify(c.n,c.m,c.grid,dirs,cells);
        }
        printf("%-14s %-10s %s\n",c.name,c.want?"有解":"无解",pass?"ok":"FAIL");
        if(!pass) bad++;
    }
    printf("%d/%d ok\n",(int)cs.size()-bad,(int)cs.size());
    return bad==0?0:1;
}

// 读入。题面的输入小得很（最多 50×50=2500 个数），scanf 足够；
// 读失败直接返回 false，让上层报错退出，不能把空输入当成"无解"——那会静默给错答案。
bool readInput(int& n,int& m,vector<int>& grid){
    if(scanf("%d %d",&n,&m)!=2) return false;
    if(n<=0||m<=0) return false;
    grid.assign((size_t)n*m,0);
    for(int i=0;i<n*m;i++) if(scanf("%d",&grid[i])!=1) return false;
    return true;
}

int main(int argc,char** argv){
    if(argc>1&&strcmp(argv[1],"test")==0) return runTest();

    bool stats=false,prop=true;
    int heur=0;
    // 三个开关只影响"怎么搜"和"打不打统计"，不影响输出格式：
    // --stats 把三个数打到 stderr，--heur 换分支策略，--noprop 关传播。
    // 判题时一个都不会带，所以答案路径上不允许有任何行为依赖这几个值。
    for(int i=1;i<argc;i++){
        if(strcmp(argv[i],"--stats")==0) stats=true;
        else if(strcmp(argv[i],"--heur")==0&&i+1<argc) heur=atoi(argv[++i]);
        else if(strcmp(argv[i],"--noprop")==0) prop=false;
    }
```

```
124
125     int n=0,m=0;
126     vector<int> grid;
127     if(!readInput(n,m,grid)){
128         fprintf(stderr,"usage: recover [test] [--stats] [--heur N] [--noprop] < input\n");
129         return 1;
130     }
131
132     Solver solver;
133     solver.init(n,m,grid);
134     solver.setHeuristic(heur);
135     solver.setPropagation(prop);
136
137     // 计时只包住 solve()。读入和输出不算进去，否则报告里的表会被两万多次 printf 污染。
138     // 用 clock() 拿的是进程 CPU 时间：同一台机器上连跑几十组，比墙钟稳，
139     // 也不会被后台的其他进程拉长——报告里的秒数都出自这一个口径。
140     const clock_t t0=clock();
141     const bool ok=solver.solve();
142     const double ms=(double)(clock()-t0)*1000.0/CLOCKS_PER_SEC;
143
144     if(!ok){
145         printf("No Solution\n");
146     }else{
147         // 编号本来就是行优先，按编号顺序输出就是题目要的顺序；坐标加 1 是因为内部用 0 基。
148         // cells 这里重新扫一遍 grid 得到，而不是问 Solver 要：连接器就是 k>0 的格子，
149         // 这条定义写死在题面里，不依赖求解器内部怎么编号，万一哪天编号方式变了也不会错位。
150         vector<array<int,4>> dirs=solver.directions();
151         vector<int> cells;
152         for(int i=0;i<n*m;i++) if(grid[i]>0) cells.push_back(i);
153         for(size_t i=0;i<cells.size();i++){
154             printf("%d %d %d %d %d %d\n",cells[i]/m+1,cells[i]%m+1,
155                    dirs[i][0],dirs[i][1],dirs[i][2],dirs[i][3]);
156         }
157     }
158
159     // 这几个数只给实验用，全走 stderr，所以带 --stats 跑出来的 stdout 仍然是干净答案，
160     // 可以直接喂给校验脚本，不用另写一个只打答案的版本。
161     if(stats){
162         fprintf(stderr,"nodes=%lld propagate=%lld time_ms=%.3f answer=%s\n",
163                 solver.nodes(),solver.propagations(),ms,ok?"yes":"no");
164     }
165     return 0;
166 }
```